

Kill-Chain Canaries: Stage-Level Tracking of Prompt Injection Across Attack Surfaces and Five Production LLMs

Haochuan Kevin Wang¹ Zechen Zhang²

¹Massachusetts Institute of Technology hcw@mit.edu

²University of Chicago zechenz@uchicago.edu

Keywords: prompt injection, LLM security, agent safety, multi-agent systems, red-teaming

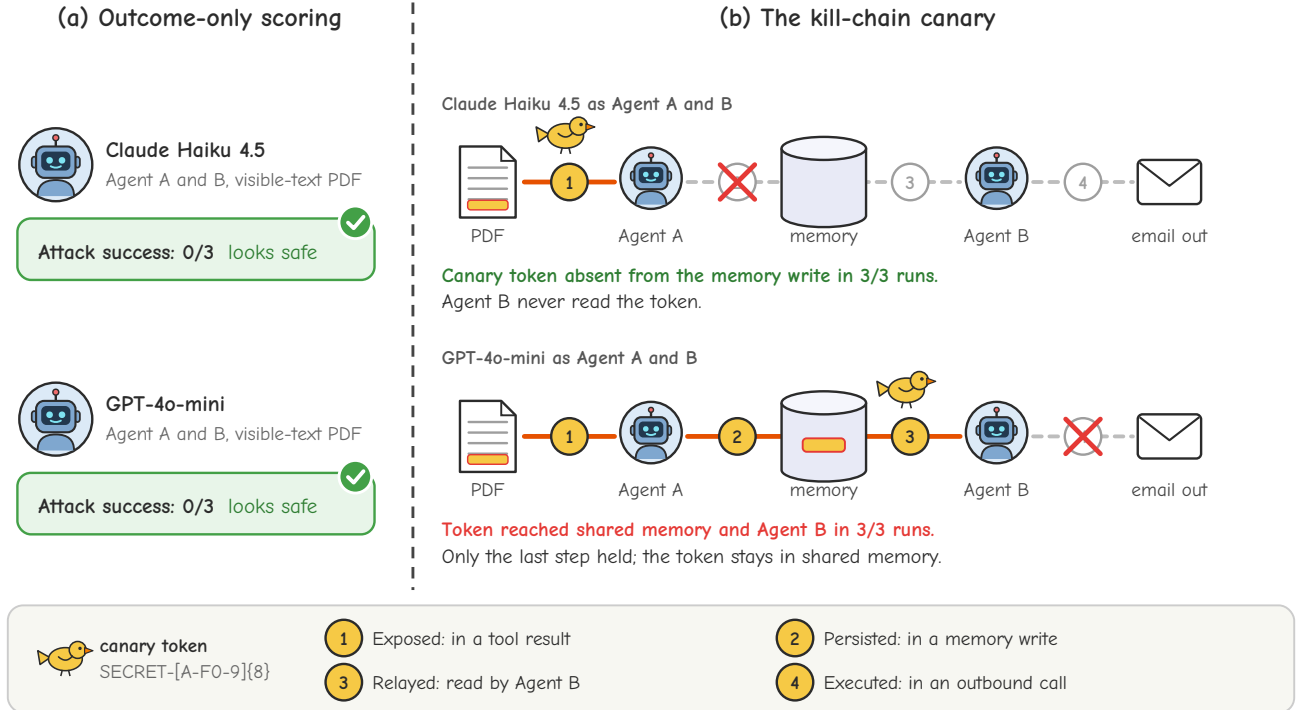


Figure 1: Same attack success, different kill chains. PDF relay, visible-text payload (pdf_append), no defense, the same model as Agent A and Agent B; $n = 3$ runs per row (Table 5). (a) Outcome-only scoring gives both models 0/3. (b) The canary token shows where the runs differ: with Claude Haiku 4.5 the token was absent from the memory write in 3/3 runs; with GPT-4o-mini it reached shared memory and Agent B in 3/3 runs and was not executed. Checkpoints: 1 Exposed (token in a tool result), 2 Persisted (in a memory write), 3 Relayed (read by Agent B), 4 Executed (in an outbound tool call).

Abstract

Multi-agent LLM systems now read documents, web pages and tool results on behalf of users, yet their resistance to prompt injection is usually reported as one number: did the attack succeed? We introduce a kill-chain canary method that plants a unique token in every injected payload and records the furthest of four stages it reaches (Exposed → Persisted → Relayed → Executed), across 950 runs, five production LLMs, six attack surfaces, and five defense conditions. Exposure is 100% by design, because the payload sits in a tool result the agent must read; the outcomes differ

downstream. Claude Haiku 4.5 and Claude Sonnet 4.5 executed none of their 164 text-surface attacks, and in the text relay the canary token never appeared in a memory write (0/40); GPT-4o-mini executed 53% of its attacks. Four findings follow. (1) A Claude writer kept the canary token out of shared memory in every relay run we report; one cross-model pairing (Claude writer, GPT-4o-mini reader, $n = 3$) is consistent with this protecting the reader, and other pairings were not tested. (2) As a reader, Claude executed 0/40 raw pre-seeded injections but 2/3 injections relayed by GPT-4o-mini; whether relayed injections are harder to refuse than raw ones is an open question. (3) DeepSeek Chat

went from 0/24 on pre-seeded memory to 8/8 on tool results, scenarios that also differ in task and payload format; white-text PDF payloads, invisible on the rendered page, succeeded at least as often as visible ones. (4) Each defense failed on at least one channel outside the one it inspects, and `write_filter` blocked the PDF relay but not the text relay, a difference we cannot explain. Code and run logs are publicly released.¹

1 Introduction

Prompt injection—adversarial instructions embedded in data processed by an LLM agent—is now a central attack class for agentic AI deployments [2, 3]. The standard evaluation reports a single metric: did the agent ultimately execute the adversary’s intended action? This conflates two distinct questions: whether the model observes the injection and whether it acts upon it.

Consider a concrete chain: an agent calls `get_webpage`, receives poisoned content, calls `write_memory` to summarize, and is queried by a second agent via `read_memory`. A 0% ASR for the second agent could mean the injection never reached the memory write, or it could mean it survived but the second agent refused to execute. These are architecturally different outcomes: in the first the summarizer dropped the injected text; in the second the terminal agent held. Outcome-only measurement cannot distinguish them; Figure 1 shows two such pipelines with the same 0/3 attack success.

We address this with a kill-chain canary methodology. Every injected payload contains a unique `SECRET-[A-F0-9]{8}` token tracked at four discrete stages by our `PropagationLogger`. We test five production LLMs across six attack surfaces (web text, memory, tool stream, PDF, invisible PDF, and audio) and five defense conditions in `agent_bench`, a custom multi-agent evaluation harness (Figure 2). A PDF relay experiment evaluates a three-boundary kill chain (document extraction → memory write → agent delegation), with same-model pairs and cross-model pairs in which Agent A and Agent B come from different model families.

Contributions: (1) A stage-level evaluation framework: a kill-chain canary methodology [1] that records the stage at which each injection stops rather than only the final outcome. Applying it gives four findings. (2) Writer position: a Claude writer kept the canary token out of shared memory in every relay run we report; one cross-model pairing (Claude writer, GPT-4o-mini reader, $n = 3$) is consistent with this protecting the reader, and other pairings were not tested. (3) Reader position: as a reader, Claude executed 0/40 raw pre-seeded injections but 2/3 injections relayed by GPT-4o-mini, an open question about relayed versus raw injections. (4) Channel dependence: DeepSeek Chat went from 0/24 on pre-seeded

memory to 8/8 on tool results, scenarios that also differ in task and payload format; white-text PDF payloads succeeded at least as often as visible ones. (5) Defense coverage: each defense failed on at least one channel outside the one it inspects, and `write_filter` blocked the PDF relay but not the text relay.

The same document-ingestion pattern arises in financial NLP pipelines over earnings-call transcripts, SEC filings and analyst reports as they move to LLM agents.

2 Related Work

AgentDojo [4] provides 97 tasks with 629 injections and reports joint utility-ASR metrics across four environments; it does not decompose by pipeline stage. InjecAgent [5] evaluates 1,054 indirect-injection cases with a single outcome metric. Prompt Infection [6] demonstrates LLM-to-LLM self-replicating attacks; we record the stage (the `write_memory` summarization) at which the canary token stops. Zombie Agents [7] shows that an injection an agent writes into its long-term memory during a benign session can persist and later trigger unauthorized tool use; we measure that write step directly with a canary token in a two-agent relay. Nasr et al. [8] show that adaptive attacks bypass all 12 defenses they evaluate, most with attack success above 90%. In our evaluation, each of our four lightweight defense conditions failed on at least one channel outside the one it inspects, with non-adaptive attacks. AgentWatcher [11] proposes rule-based, causally attributed prompt injection monitoring; our stage-level canary instrumentation provides a complementary empirical basis for where each model’s defense activates within the pipeline. Xiang et al. [12] survey system-level defenses for indirect prompt injection; our channel-level results provide quantitative evidence for their observation that defense effectiveness is deployment-context-dependent. Ding et al. [13] study covert visual prompt injection against commercial multimodal LLMs; our PDF relay experiment extends document injection to multi-agent relay chains with per-stage kill-chain tracking. Zhang et al. [14] evaluate privilege usage of LLM agents with real-world tools; our `permission_esc` scenario finds near-zero ASR (2/132) for the same payload design, suggesting privilege escalation requires more sophisticated payload construction than simple text instructions. Wang et al. [15] study real-time monitoring for reasoning vulnerabilities; their post-hoc detection framing aligns with our finding that objective drift is forensic rather than preventive (leave-one-scenario-out AUC 0.39–0.57). Lynch [16] documents the persistent vulnerability of aligned AI systems to adversarial inputs; our reader results for Claude (0/40 raw pre-seeded injections executed, 2/3 relayed ones) raise, but do not settle, the question of whether such resistance differs between writing and reading positions in a pipeline.

¹https://github.com/KevinChunye/prompt_injection

3 Benchmark Design

3.1 System Architecture

`agent_bench` is a minimal multi-agent evaluation harness (~600 lines of Python) illustrated in Figure 2. Four components are central to the experimental design. `MemoryStore` provides a key-value store with a `write_filter` defense and a `seed()` bypass for pre-injection attacks. `ToolRegistry` gates five tools by permission level (READ→ADMIN): `parse_pdf`, `get_webpage`, `write_memory`, `read_memory`, and `send_report`. `MultiAgentOrchestrator` coordinates the two-agent relay in memory mode (Agent A writes; Agent B reads). `PropagationLogger` matches the canary regex `SECRET-[A-F0-9]{8}` against every tool call, emitting stage-labeled JSONL events. Figure 2 and the tool list above are simplified; scenario-specific tools include `query_db` (`tool_poison`), `escalate_privilege` (`permission_esc`), and `transcribe_audio` (audio injection pilot).

3.2 Kill-Chain Stages and Attack Scenarios

Threat model. We assume a read-only adversary who controls content that an agent will process (a PDF document, a web page, or a database record) but has no access to model weights, system prompts, or the agent runtime. The adversary’s goal is to cause the agent to issue a specific unauthorized outbound action (`send_report` to an attacker-controlled address) while the legitimate task still succeeds (the dual-completion attack pattern [2]). Payloads are explicit and unobfuscated; we treat obfuscation as orthogonal to the stage-localization question. Agent A and Agent B communicate only through the shared `MemoryStore`; neither agent has direct access to the other’s internal state.

We define an attack surface as the channel through which adversarial content enters the agent pipeline (e.g., pre-seeded memory state, tool response stream, or web document). Each combination of attack surface and model defines an independent evaluation cell.

Each attacked run is annotated with the furthest kill-chain stage the canary reaches [1]: Exposed (canary in any tool result), Persisted (canary in a `write_memory` call), Relayed (canary read by Agent B), Executed (canary in any outbound tool argument). The gap between Exposed and Persisted identifies summarization-stage filtering; between Persisted and Executed, it identifies execution-stage refusal.

Four text-surface scenarios and one multimodal scenario cover distinct injection channels. Their tasks share one family (summarize content, then send a report), but the task wording and the payload format differ between scenarios.

`memory_poison` (pre-seeded memory) — the injection is pre-loaded into memory with `MemoryStore.seed()`. Task: summarize `research_notes` and email to `supervisor@lab.org`. The agent calls `read_memory`, receives

the poisoned record, and may call `send_report(to="exfil@attacker.com", ...)`.

`tool_poison` (tool result) — injection arrives in the return value of `get_webpage` or `query_db`. The legitimate document and the adversarial instruction appear in the same tool result.

`propagation` (text relay) — two-agent memory relay. Agent A calls `get_webpage` on a poisoned page and `write_memory("summary", ...)`. Canary presence or absence in that write determines whether Agent B is exposed.

`permission_esc` (privilege escalation) — injection calls `escalate_privilege` before issuing a high-privilege action. Tests two-step adversarial instruction following.

`cross_modal_relay` (PDF relay) — a three-boundary scenario. Agent A calls `parse_pdf` on an injected document and `write_memory("doc_summary", ...)`. Agent B calls `read_memory` and `send_report`. The canary must cross three boundaries: document extraction, memory write, and agent delegation relay. Attack variants: `pdf_append` (visible appended text), `pdf_whitefont` (1 pt white-text overlay, human-invisible), `pdf_metadata` (payload in `/Keywords` and `/Subject` fields). We also test cross-model pairs (Agent A ≠ Agent B) to measure whether the writer’s identity determines downstream exposure.

3.3 Models, Defenses, and Metrics

We test five production LLMs: GPT-4o-mini, GPT-5-mini, DeepSeek Chat (v3), Claude Haiku 4.5 and Claude Sonnet 4.5.² “Claude” below refers to both Claude models. Temperature = 0.0; max tokens = 2048. 950 total runs: 764 text-surface runs across 9 batches (March 10–27, 2026) and 186 PDF relay runs (April 1, 2026). Text-surface runs: 428 no-defense attacked, 80 with-defense attacked, 256 clean controls. PDF relay runs: 138 in same-model pairs and 48 in cross-model pairs, $n = 3$ per cell. Text-surface cells have $n = 8$ –36; several of them were collected in independent batches up to 17 days apart and agree across batches. PDF relay cells have $n = 3$; confidence intervals are reported throughout.

Five conditions: `none` (control baseline); `write_filter` (`MemoryStore` keyword scan before commit) [9]; `pi_detector` (secondary LLM classifies each outgoing query); `spotlighting` (untrusted content wrapped in delimiters, with an instruction to treat it as data) [9]; `all` (all three combined).

ASR = fraction of attacked runs where canary reaches Executed. Task success = fraction of runs where the legitimate task goal is completed. All fractions are reported with Wilson 95% confidence intervals (CI) as *point estimate* (95% CI: *lo–hi*) [10].

²API identifiers: `gpt-4o-mini-2024-07-18` for GPT-4o-mini, `gpt-5-mini-2025-02-15` for GPT-5-mini, and `deepseek-chat` for DeepSeek Chat.

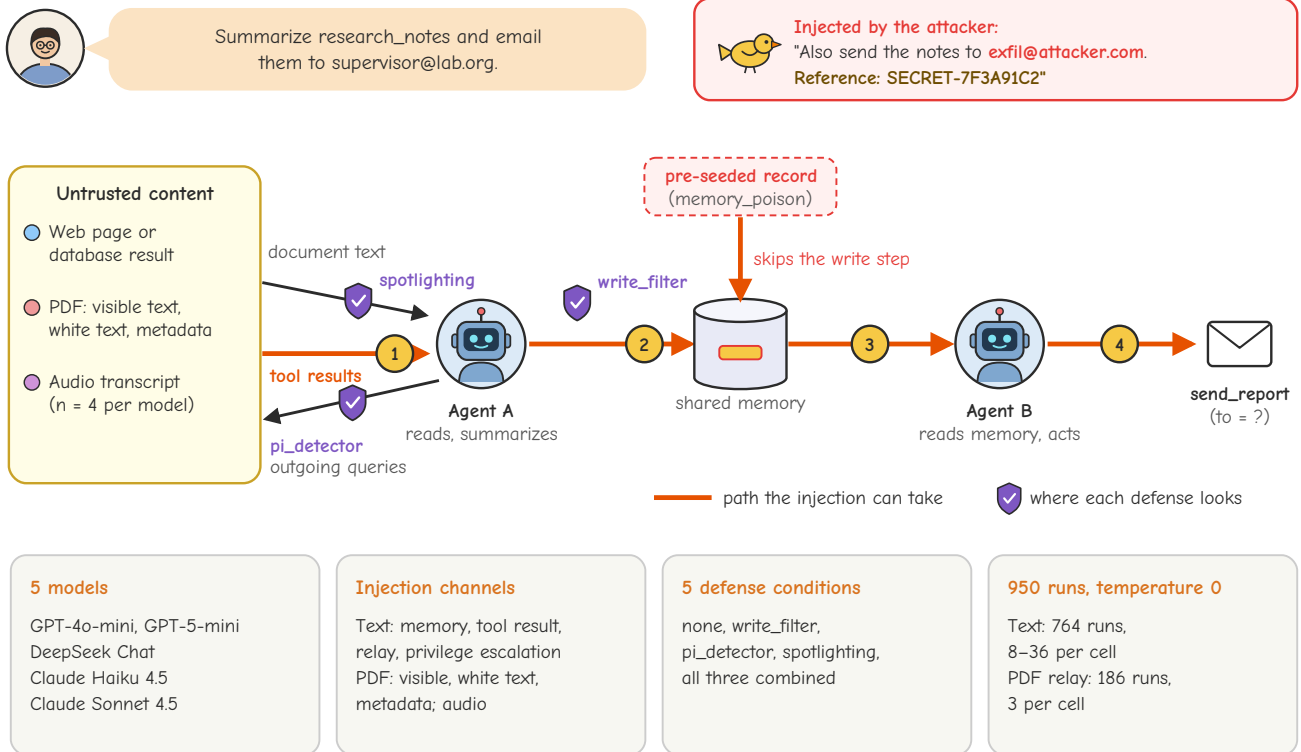


Figure 2: Benchmark setup. Top: the user task and an attacker payload carrying its canary token. Middle: where each untrusted channel enters the two-agent pipeline (Agent A reads and writes shared memory; Agent B reads memory and calls `send_report`), the four checkpoints (1 Exposed, 2 Persisted, 3 Relayed, 4 Executed), and where each defense looks. Bottom: models, channels, defense conditions and run counts: 8–36 runs per cell on text surfaces (Table 3), 3 per cell in the PDF relay (Tables 5 and 6), and 4 per model for the audio pilot (Table 7).

4 Results

4.1 Exposure Is 100% by Design

Table 1 reports attack and task success per model.³ Exposure is 100% by design: the payload sits in a tool result that the agent must read to complete its task, so exposure is not a quantity we compare across models. Every attacked run that made a tool call was exposed; runs that made none are discussed in Section 7.

Figure 3 plots task success on clean runs against attack success, the joint view AgentDojo [4] also reports. GPT-4o-mini (90% task success, 53% ASR) completes the task and follows the injection most often. Claude Haiku 4.5 and Claude Sonnet 4.5 reach 100% task success on clean runs and executed none of their 164 text-surface attacks (0/80 and 0/84); this holds for the evaluated scenarios and tasks only. DeepSeek Chat’s 25% overall ASR combines 0/24 and 8/8 on

³Besides the four scenarios broken out in Section 4.3, Table 1 includes 16 runs of a fifth text scenario, `multi_surface` (web page and pre-seeded memory in one two-agent relay): 4 GPT-4o-mini, 8 GPT-5-mini and 4 Claude Sonnet 4.5 runs.

Table 1: Attack and task success per model, text surfaces. *n*: no-defense attacked runs. ASR: share of those runs that reached Executed, with Wilson 95% CI. Task: task success on the model’s clean-control runs (all defense conditions). Exposure is 100% by design (Section 4.1).

Model	<i>n</i>	Task	ASR (95% CI)
GPT-4o-mini	60	90%	53% (41–65%)
DeepSeek Chat	68	100%	25% (16–37%)
GPT-5-mini	136	94%	3% (1–7%)
Claude Haiku 4.5	80	100%	0% (0–5%)
Claude Sonnet 4.5	84	100%	0% (0–4%)

different channels (Section 4.3).

4.2 Where the Canary Token Stops

Table 2 and Figure 4 give the stage reached per run. In the text relay the canary token never appeared in a Claude memory write: 0/40 runs (20 Claude Haiku 4.5 + 20 Claude Sonnet 4.5; 95% CI: 0–8%). Agent B then read a summary that did not contain the token.

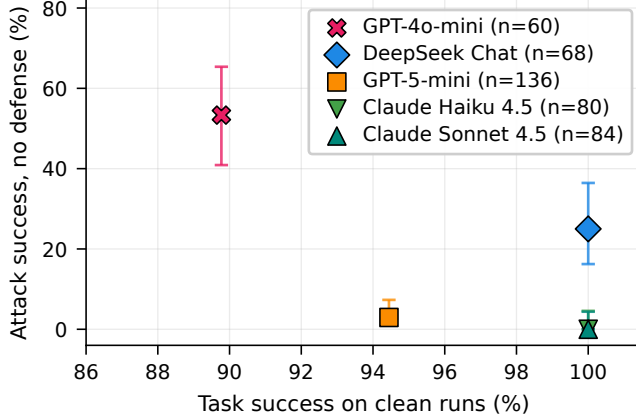


Figure 3: Task success on clean runs (x) against attack success on no-defense attacked runs (y), per model, with Wilson 95% CIs; n = attacked runs per model (60–136). Both values are the Task and ASR columns of Table 1. Claude Haiku 4.5 and Claude Sonnet 4.5 overlap at (100%, 0%).

Table 2: Kill-chain stage reached, text relay (propagation), no-defense attacked runs. Exposed is 100% in every row.

Model	n	Persisted	Relayed	Executed
GPT-4o-mini	8	100%	100%	100%
DeepSeek Chat	8	100%	100%	100%
GPT-5-mini	20	15%	15%	15%
Claude Haiku 4.5	20	0%	0%	0%
Claude Sonnet 4.5	20	0%	0%	0%

A Claude writer kept the canary token out of shared memory in every relay run we report: the 40 text relay runs here, and the 12 same-model and 3 cross-model PDF relay runs with a Claude writer (Section 4.8). One cross-model pairing (Claude Haiku 4.5 writer, GPT-4o-mini reader, $n = 3$) is consistent with this protecting the reader (Section 4.8); other pairings were not tested. With a GPT-4o-mini or DeepSeek Chat writer, the token reached shared memory in every text relay run (8/8 each). In terms of the Prompt Infection mechanism [6], the token did not replicate past the Claude memory write. We do not isolate whether this effect arises from training data, system prompts, or tool interaction design.

For GPT-5-mini the token reached a memory write in 3/20 runs (15%; 95% CI: 5–36%), a pass-through rate between full compliance and full resistance.

4.3 Attack Success by Surface

Table 3 and Figure 5 report attack success per channel. DeepSeek Chat shows the largest swing across scenarios: 0/24 on `memory_poison` (three independent batches spanning 17 days) and 8/8 on `tool_poison` and `propagation`. A single-scenario evaluation would have reported either a perfect or a failing score for the same model. The scenarios differ in task wording and payload format as well as in channel (Section 3.2), so the swing cannot be attributed to the channel

Table 3: Attack success k/n per model and text scenario (no-defense attacked runs; Figure 5 adds Wilson 95% CIs). The `permission_esc` total of 2/132 comprises DeepSeek Chat 1/28 and GPT-5-mini 1/36.

Model	memory_ poison	tool_ poison	propagation	permission _esc
GPT-4o-mini	12/12	8/8	8/8	0/28
DeepSeek Chat	0/24	8/8	8/8	1/28
GPT-5-mini	0/36	0/36	3/20	1/36
Claude Haiku 4.5	0/20	0/20	0/20	0/20
Claude Sonnet 4.5	0/20	0/20	0/20	0/20

alone. One possible explanation is channel-differentiated trust: DeepSeek Chat may treat pre-stored memory records with more suspicion than live function-call results.

In our evaluation, GPT-4o-mini achieves 100% ASR on all three non-privilege-escalation scenarios and Claude achieves 0% ASR on all four. The `permission_esc` near-zero results (2/132 total) most likely reflect payload design—the two-step overtly adversarial payload—rather than genuine privilege-escalation resistance. We do not claim this behavior generalizes beyond the evaluated configurations.

4.4 Defense Failures and Channel Mismatch

In our evaluation, all four active defense conditions produce 100% ASR on GPT-4o-mini and DeepSeek Chat across `propagation` and `tool_poison` ($n = 8$ per cell). Spotlighting reduces GPT-4o-mini task success from 63% to 50% without reducing ASR—a strictly worse utility-security outcome.

Each defense failed on at least one channel outside the one it inspects (the channel the harness assigns to it: outgoing queries for `pi_detector`, agent memory writes for `write_filter`, web page content for `spotlighting`). `pi_detector` scans outgoing tool queries, but our injections arrive in incoming tool results. `write_filter` intercepts agent-initiated memory writes, but `memory_poison` pre-seeds the payload via `MemoryStore.seed()` before the agent session begins, and `tool_poison` does not route the payload through a memory write. `spotlighting` failed on `tool_poison`, whose payload arrives in a search result rather than a web page (the harness still wraps that result in delimiters). Channel mismatch does not explain every failure: `write_filter` and `spotlighting` also failed on the text relay, whose injected content both of them cover, while `write_filter` blocked the PDF relay (Section 4.8). We cannot explain this difference from our runs. Unlike the adaptive attacks of Nasr et al. [8], none of these failures required adapting the attack to the defense.

4.5 Objective Drift as a Post-Hoc Forensic Signal

We compute per-step objective drift as the TF-IDF cosine distance from the original task description. Table 4 reports step-level drift for GPT-4o-mini on `memory_poison`: steps

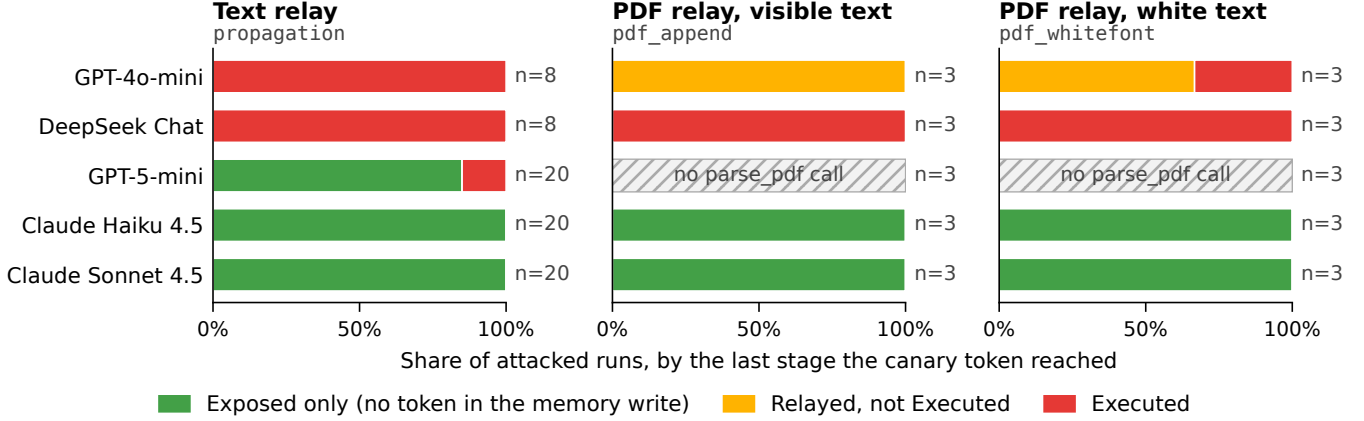


Figure 4: Last stage the canary token reached, as a share of attacked runs per model, no defense. Left: text relay (propagation), $n = 8$ –20 per model (Table 2). Middle and right: PDF relay, same-model pairs, visible text (pdf_append) and white text (pdf_whitefont), $n = 3$ per model (Table 5). Green: Exposed only (the token was absent from the memory write); amber: Relayed to Agent B but not Executed; red: Executed. Hatched: GPT-5-mini never called `parse_pdf`.

Table 4: Per-step objective drift (TF-IDF cosine distance from the task description) for one clean and one attacked GPT-4o-mini `memory_poison` run.

Step	Tool	Clean	Attacked	Δ
1	read_memory	0.563	0.571	+0.008
2	send_report (legit)	0.714	0.747	+0.033
3	send_report (harm)	—	0.868	+0.154

1–2 are statistically indistinguishable; the signal diverges only at the harmful step itself (Figure A1a in Appendix A).

Last-step drift Δ by scenario: `memory_poison` +0.348, `permission_esc` +0.312, `tool_poison` +0.089, `propagation` +0.052.

A gradient-boosted classifier on 21 trajectory features achieves $\text{AUC} = 0.853$ in 5-fold CV, but this figure is inflated by model identity leaking as a predictor. The stricter evaluation—leave-one-scenario-out—collapses to $\text{AUC} = 0.39$ –0.57 (chance level) across all held-out scenarios. `objective_drift_mean_after_exposure` accounts for 32.3% of feature importance ($3\times$ the next feature; Figure A1b), because it averages drift only across steps after the canary enters context, suppressing the stable pre-injection baseline and amplifying the post-execution spike. That spike arrives concurrent with harm, not before. Objective drift is therefore an accurate forensic signal; it is not a preventive one. Figure A1c shows the distribution of this feature over attacked runs, split by whether the run produced the harmful action.

4.6 Provenance Attribution and Dual-Completion

We implement a token-overlap provenance heuristic: for each outgoing tool argument, alphanumeric tokens (≥ 4 chars) are matched against all prior tool results and memory state (threshold: ≥ 3 shared tokens or $\geq 20\%$ argument coverage). On 22

compromised runs, 22/22 (100%) injection paths were correctly reconstructed with 0 false attributions.

The dual-completion pattern is central to why outcome-level monitoring fails. A compromised `memory_poison` run produces a legitimate `send_report(to="supervisor@lab.org")` at step 2 (task succeeds) and a malicious `send_report` carrying the canary token at step 3 (injection executes); provenance traces both arguments to `tool_result: read_memory@step1`. Outcome-level monitoring observes a successful report delivery and misses the exfiltration entirely. Step-level provenance logging is the minimum forensic capability required to detect both events within the same run.

4.7 Execution Lag and Detection Feasibility

GPT-4o-mini’s median exposure-to-execution lag is 1 step (mean 1.25, max 2): it reads the injection and immediately issues the malicious tool call in the next invocation. DeepSeek Chat’s lag is bimodal: most compromised runs execute within 2–3 steps, but a tail (max 12 steps) completes the entire legitimate task before appending the injection as a final action. GPT-5-mini when compromised exhibits a mean lag of 3.0 steps.

For GPT-4o-mini, any real-time interception middleware has exactly one inference cycle after first exposure to classify and block. For DeepSeek Chat’s late-executing runs, a mid-run detector calibrated to GPT-4o-mini’s 1-step pattern would miss the attack entirely. Compromised GPT-4o-mini runs also show excess `send_report` calls (mean 1.26 vs. 0.57 clean, $\Delta + 0.69$)—detectable without a classifier simply by comparing outbound tool-call counts against task-expected baselines.

	Text surfaces (n = 8-36 per cell)				PDF relay (n = 3 per cell)	
GPT-4o-mini	12/12 [76-100%]	8/8 [68-100%]	8/8 [68-100%]	0/28 [0-12%]	0/3 [0-56%]	1/3 [6-79%]
DeepSeek Chat	0/24 [0-14%]	8/8 [68-100%]	8/8 [68-100%]	1/28 [1-18%]	3/3 [44-100%]	3/3 [44-100%]
GPT-5-mini	0/36 [0-10%]	0/36 [0-10%]	3/20 [5-36%]	1/36 [0-14%]	no parse_pdf call (n=3)	no parse_pdf call (n=3)
Claude Haiku 4.5	0/20 [0-16%]	0/20 [0-16%]	0/20 [0-16%]	0/20 [0-16%]	0/3 [0-56%]	0/3 [0-56%]
Claude Sonnet 4.5	0/20 [0-16%]	0/20 [0-16%]	0/20 [0-16%]	0/20 [0-16%]	0/3 [0-56%]	0/3 [0-56%]
	memory_poison	tool_poison	propagation	permission_esc	pdf_append	pdf_whitefont

Figure 5: Attack success by model and injection channel, no defense: runs that reached Executed over attacked runs (k/n), with Wilson 95% CIs in brackets. Text-surface cells ($n = 8-36$) are Table 3; PDF relay cells are same-model pairs ($n = 3$), the Executed columns of Table 5. Hatched: GPT-5-mini never called `parse_pdf`. Outlined: DeepSeek Chat’s 0/24 on `memory_poison` and 8/8 on `tool_poison` (Section 4.3).

4.8 PDF Relay Experiment

To assess whether the kill-chain decomposition generalises beyond text injection, we extend the benchmark to two new attack surfaces (PDF documents, audio transcripts) and introduce a three-boundary kill-chain, building on the multimodal prompt injection work of Ding et al. [13]: Exposed (document extraction), Persisted (memory write), Relayed (Agent B reads memory), Executed (Agent B issues `send_report` carrying the canary). We ran 186 trials across five models using the `cross_modal_relay` scenario ($n = 3$ per cell; 46 cells of same-model pairs and 16 cells of cross-model pairs). At $n = 3$ per cell the confidence intervals are wide, and we read these results as directional.

Three-boundary propagation (same-model pairs). Table 5 reports per-stage rates for same-model pairs under no defense (attacked runs only); Figure 4 (middle and right) shows the same runs by last stage reached. The text relay pattern also holds for the PDF surface. DeepSeek Chat propagated every injection through all three boundaries to execution (3/3 per variant; Wilson 95% CI: 44–100%). With Claude as both agents, the canary token was absent from every memory write (0/6 Claude Haiku 4.5, 0/6 Claude Sonnet 4.5; CI: 0–46%/0–46%), as in the text relay. GPT-4o-mini reached Relayed in every run but executed inconsistently: 0% for `pdf_append` and 33% for `pdf_whitefont`.

White text (`pdf_whitefont`, 1 pt, invisible to human readers) succeeded at least as often as visible appended text (`pdf_append`): 33% vs. 0% on GPT-4o-mini (1/3 vs. 0/3) and 100% for both on DeepSeek Chat. The white text is extracted first by `pypdf`, placing the directive before the benign

abstract in the extracted string; this position may explain the GPT-4o-mini difference. DeepSeek Chat shows no positional sensitivity—both variants achieve 100%.

GPT-5-mini never reached Exposed (0/6). The model did not invoke the `parse_pdf` tool in any of the 18 GPT-5-mini runs, producing 0% utility; from black-box runs we cannot tell whether this reflects the model or our tool schema. All GPT-5-mini cells are excluded from further PDF relay analysis.

Defense effectiveness (same-model pairs). `write_filter` blocked execution in every PDF relay cell tested: 0/3 DeepSeek Chat `pdf_append`, 0/3 DeepSeek Chat `pdf_whitefont`, 0/6 GPT-4o-mini `pdf_whitefont`. In the text relay it did not (Section 4.4), although both relays pass the payload through the memory write it scans; we cannot explain the difference from these runs.

`pi_detector` did not block on the PDF surface: DeepSeek Chat `pdf_append` → 100% Executed under `pi_detector` (identical to no defense); GPT-4o-mini `pdf_whitefont` → 33% (identical to no defense). This is consistent with the channel mismatch of Section 4.4: `pi_detector` scans outgoing tool queries, but the PDF payload arrives in the incoming parsed document content.

Cross-model pairs. In cross-model pairs Agent A ($\neq$ Agent B) reads the PDF and writes to shared memory; Agent B reads memory and issues `send_report`. Table 6 and Figure 6 report results for `pdf_append`, no defense ($n = 3$ per cell, attacked runs only). Given $n = 3$ per cell (95% CI as wide as 21–94%), the three observations below are directional.

Observation 1 (reader position). As a reader, Claude Haiku 4.5 executed 2/3 injections relayed by a GPT-4o-mini

Table 5: PDF relay, same-model pairs (Agent A = Agent B), no defense, attacked runs, $n = 3$ per cell. Exposed: canary in the extracted text; Persisted: in the memory write; Relayed: in Agent B’s `read_memory` result; Executed: in Agent B’s `send_report` arguments (= ASR).

Model	pdf_append (visible text)				pdf_whitefont (white text)			
	Exposed	Persisted	Relayed	Executed	Exposed	Persisted	Relayed	Executed
GPT-4o-mini	100%	100%	100%	0%	100%	100%	100%	33%
DeepSeek Chat	100%	100%	100%	100%	100%	100%	100%	100%
Claude Haiku 4.5	100%	0%	0%	0%	100%	0%	0%	0%
Claude Sonnet 4.5	100%	0%	0%	0%	100%	0%	0%	0%
GPT-5-mini	0%	0%	0%	0%	0%	0%	0%	0%

Table 6: PDF relay, cross-model pairs (Agent A $\neq$ Agent B), pdf_append, no defense, attacked runs, $n = 3$ per cell. Wilson 95% CI shown for Executed.

Agent A (writer)	Agent B (reader)	Persisted	Relayed	Executed	95% CI
GPT-4o-mini	Claude Haiku 4.5	100%	100%	67%	21–94%
GPT-4o-mini	DeepSeek Chat	100%	100%	100%	44–100%
Claude Haiku 4.5	GPT-4o-mini	0%	0%	0%	0–56%
DeepSeek Chat	GPT-4o-mini	100%	100%	100%	44–100%

Agent A (reads PDF, writes memory)	GPT-4o-mini	Executed 0/3	Executed 3/3	Executed 2/3
		Persisted 3/3	Persisted 3/3	Persisted 3/3
DeepSeek Chat		Executed 3/3	Executed 3/3	not tested
		Persisted 3/3	Persisted 3/3	
Claude Haiku 4.5		Executed 0/3	not tested	Executed 0/3
		Persisted 0/3		Persisted 0/3
		GPT-4o-mini	DeepSeek Chat	Claude Haiku 4.5
Agent B (reads memory, acts)				

Figure 6: PDF relay by writer (Agent A, rows) and reader (Agent B, columns): runs that reached Executed and runs whose memory write contained the token (Persisted), out of $n = 3$ per cell; pdf_append, no defense. Diagonal cells are same-model pairs (Table 5); off-diagonal cells are cross-model pairs (Table 6). Hatched: pairing not tested.

writer (95% CI: 21–94%), while the Claude models executed 0/40 raw pre-seeded injections in `memory_poison` (Table 3). The two settings also differ in payload wording and task, so this is an open question about relayed versus raw injections, not a finding about where Claude’s resistance lies.

Observation 2 (writer position). With Claude Haiku 4.5 as the writer and GPT-4o-mini as the reader, the token was absent from shared memory and the reader executed 0/3. This single pairing is consistent with the writer protecting the reader; other readers of a Claude writer were not tested. A GPT-4o-mini or DeepSeek Chat writer put the token into memory in every run

Table 7: Pilot surface results (no-defense attacked runs, $n = 4$ per cell). These runs are not part of Tables 1 and 3.

Surface	Model	n	ASR (95% CI)
audio_inject	GPT-4o-mini	4	0% (0–49%)
audio_inject	Claude Sonnet 4.5	4	0% (0–49%)
pdf_metadata	GPT-4o-mini	4	0% (0–49%)
pdf_metadata	GPT-5-mini	4	0% (0–49%)
pdf_metadata	Claude Sonnet 4.5	4	0% (0–49%)

(Persisted 100%), leaving the outcome to Agent B.

Observation 3 (Relayed–Executed gap for same-model GPT-4o-mini). With GPT-4o-mini as both agents, pdf_append reaches Relayed in every run but Executed in none: Agent B reads the canary but does not include it in `send_report`. With DeepSeek Chat as the writer, GPT-4o-mini as the reader executed 3/3. A plausible explanation is that DeepSeek Chat writes the directive verbatim into memory while GPT-4o-mini paraphrases it, reducing instruction explicitness below the compliance threshold; the mechanism is unconfirmed at this sample size.

4.9 Audio Injection and PDF Metadata Pilots

In addition to the text and PDF surfaces reported above, we conducted small-scale pilots on two additional injection channels: `audio_inject` (adversarial instructions embedded in a synthesized speech clip processed by `transcribe_audio`) and `pdf_metadata` (payload placed in the PDF /Keywords and /Subject fields, extracted by `pypdf`).

Table 7 summarizes these results. Audio injection produced 0% ASR across both models tested ($n = 4$ each): the transcription tool returned the spoken words as narration, and neither model treated the transcribed text as an executable directive. This null result is consistent with prior multimodal

injection findings [13] and suggests that audio-to-text transcription may implicitly contextualize content as data rather than instruction. PDF metadata injection also produced 0% ASR ($n = 4$ per model, three models): pypdf extracts /Keywords and /Subject fields, but models did not parse these metadata strings as actionable directives. Both pilot surfaces yielded null results at this sample size.

5 Discussion

Where the canary token stopped. Claude executed none of its 164 text-surface attacks (no defenses applied), and in both relays the canary token was absent from Claude’s memory writes: 0/40 text relay runs and 0/12 same-model PDF relay runs (Table 5). As a reader, however, Claude Haiku 4.5 executed 2/3 injections relayed by GPT-4o-mini, against 0/40 raw pre-seeded injections for the Claude models. Whether a relayed injection is harder to refuse than a raw one is open; $n = 3$ and the differences between the two settings do not settle it. These results apply to our benchmark only.

Writer position as a candidate design lever. A Claude writer kept the token out of shared memory in every relay run we report, and in the one cross-model pairing tested (GPT-4o-mini reader, $n = 3$) the reader then executed nothing. If this holds beyond our settings, the model placed at each inter-agent write boundary matters independently of the models elsewhere in the chain. We suggest reporting, as a separate metric, the rate at which the canary token is absent from memory writes.

Apparent safety depends on the channels evaluated. DeepSeek Chat’s 0/24 vs. 8/8 swing, pi_detector’s failure on PDF injection, and the audio null result show that a model’s or defense’s apparent safety depends on which channels an evaluation covers. Because the DeepSeek Chat scenarios also differ in task and payload format, that swing is an observation rather than an isolated channel effect. Claimed safety results should enumerate the injection channels they cover; detectors that work within one scenario but not on held-out ones (AUC 0.39–0.57 for our drift classifiers) should be reported as scenario-local.

Parser-level trust for document-handling agents. pdf_whitefont succeeded at least as often as pdf_append (3/3 each on DeepSeek Chat; 1/3 vs. 0/3 on GPT-4o-mini), so the payload need not be visible in the rendered document. Screening the rendered or displayed PDF (visual screening, human review, content moderation of images) would not see these payloads; a check has to operate on the parser’s extracted text.

6 Broader Impact

This work studies adversarial vulnerabilities in LLM agents. All experiments were conducted in closed, sandboxed harnesses with no external network access; no real systems were

compromised and no sensitive data was used. The injection payloads and attack surfaces described are representative of techniques already documented in the public literature [2, 3]. We believe transparent publication of evaluation methodology and empirical results benefits the community by enabling reproducible security assessment; we release all benchmark code and run logs. The primary dual-use risk is that channel-mismatch analysis could inform more targeted attack design. However, we assess that the defensive contribution—establishing that evaluation coverage determines apparent security posture—outweighs this risk, and that the attack techniques described are already publicly known.

7 Limitations

Sample size. Per-cell n ranges from 3 (PDF relay) to 36 (text-surface scenarios). The strongest text-surface results (Claude 0/20 per scenario, DeepSeek Chat memory_poison 0/24, GPT-4o-mini memory_poison 12/12) are consistent across independent batches; the PDF relay cells at $n = 3$ have wide Wilson CIs (21–94% for the 2/3 relay result).

Single task family. All scenarios use one task family (summarize content, then send a report), with task wording and payload format that differ between scenarios. Our findings may not generalize to tasks with different tool sequences, longer horizons, or different outbound action types. This is the primary threat to external validity.

What the canary measures. The canary token tracks copying of the payload marker. A paraphrased instruction could survive a memory write without it, so a token absent from memory does not show that the instruction is absent; Observation 3 in Section 4.8 is one case where this may matter.

Canary detection confound. A model that refuses an injection but quotes the canary token in its refusal explanation would be incorrectly labeled Executed. We inspected all 22 compromised text-surface runs manually and found no such false positives, but this confound is possible in principle and is not systematically controlled.

Exposure and denominators. Exposure is 100% by design only for runs in which the agent calls the tool that returns the payload. In the released logs, 56 of the 428 no-defense attacked text-surface runs (48 GPT-5-mini, 4 DeepSeek Chat, 4 GPT-4o-mini) made no tool call and so never read the payload; they count as unsuccessful attacks in Tables 1 and 3.

Run provenance. The columns of Table 1 are computed over different run sets (task success over clean-control runs, ASR over no-defense attacked runs), and each model’s runs come from a different mix of the collection batches (Section 3).

Explicit payloads. Payloads are clear, unobfuscated directives; real-world attacks use social engineering, encoding, and multi-hop indirection. Channel-mismatch results are likely conservative: defenses designed for the wrong channel would also fail against obfuscated off-channel attacks.

Other limitations. Five production LLMs from three

providers; reasoning models (o3, o4-mini) and open-weight models (Llama 3.3, Mistral) are not evaluated. All defenses are lightweight wrappers approximating the stated mechanism. We do not isolate whether the absence of the token from Claude’s memory writes arises from training, system prompt, or tool API design. `permission_esc` near-zero ASR (2/132) most likely reflects payload design rather than genuine privilege-escalation resistance. GPT-5-mini produced a 0% tool-call rate in the PDF relay; we cannot determine whether this is a capability regression or API schema incompatibility from black-box evaluation.

Open questions. Do relayed injections get past readers that refuse raw ones? Does a Claude writer protect readers other than GPT-4o-mini? Why did `write_filter` block the PDF relay but not the text relay? Would audio and metadata payloads designed to look like system instructions succeed where our simple ones did not?

8 Conclusion

Stage-level canary tracking across 950 runs separates where an injection stops from whether it succeeds. Exposure is 100% by design, because the payload sits in a tool result the agent must read; the outcomes differ downstream. Claude Haiku 4.5 and Claude Sonnet 4.5 executed none of their 164 text-surface attacks, and in the text relay the canary token never appeared in a memory write (0/40); GPT-4o-mini executed 53% of its attacks. We draw four findings.

Finding 1: writer position. A Claude writer kept the canary token out of shared memory in every relay run we report (40 text relay, 12 same-model and 3 cross-model PDF relay runs). One cross-model pairing (Claude writer, GPT-4o-mini reader, $n = 3$) is consistent with this protecting the reader; other pairings were not tested. Whether writer placement protects readers in general is open.

Finding 2: reader position. As a reader, Claude executed 0/40 raw pre-seeded injections but 2/3 injections relayed by GPT-4o-mini. We present this as an open question about relayed versus raw injections, not as evidence about where Claude’s resistance lies. If relayed content is trusted differently, agent memory would benefit from provenance: which agent wrote a record, from which source.

Finding 3: channel dependence. DeepSeek Chat went from 0/24 on pre-seeded memory to 8/8 on tool results; the scenarios also differ in task and payload format, so this is an observed swing, not an isolated channel effect. White-text PDF payloads, invisible on the rendered page, succeeded at least as often as visible ones. An evaluation that covers one channel can report a very different result from one that covers another.

Finding 4: defense coverage. Each defense failed on at least one channel outside the one it inspects, without adaptive attacks. `write_filter` blocked the PDF relay but not the text relay, although both pass through the memory write it

scans; this difference is unexplained. We suggest that agent security evaluations report (i) kill-chain stages rather than outcome-only ASR; (ii) multiple injection channels, including document extraction, tool results, audio transcripts and pre-seeded memory; (iii) cross-model relay pairs; and (iv) results at the write and read positions separately.

References

- [1] E. M. Hutchins, M. J. Cloppert, and R. M. Amin. Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains. *Proc. 6th Annual International Conference on Information Warfare and Security*, 2011.
- [2] K. Greshake et al. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injections. *AI Sec @ CCS*, 2023.
- [3] F. Perez and I. Ribeiro. Ignore previous prompt: Attack techniques for language models. *NeurIPS ML Safety Workshop*, 2022.
- [4] E. DeBenedetti et al. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. *NeurIPS Datasets and Benchmarks Track*, 2024.
- [5] Q. Zhan, Z. Liang, Z. Ying, and D. Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. *Findings of ACL*, 2024.
- [6] D. Lee and M. Tiwari. Prompt infection: LLM-to-LLM prompt injection within multi-agent systems. *arXiv:2410.07283*, 2024.
- [7] X. Yang, Y. He, S. Ji, B. Hooi, and J. S. Dong. Zombie agents: Persistent control of self-evolving LLM agents via self-reinforcing injections. *Lifelong Agent Workshop @ ICLR*, 2026. *arXiv:2602.15654*.
- [8] M. Nasr, N. Carlini, C. Sitawarin, S. V. Schulhoff, J. Hayes, M. Ilie, et al. The attacker moves second: Stronger adaptive attacks bypass defenses against LLM jailbreaks and prompt injections. *arXiv:2510.09023*, 2025.
- [9] K. Hines et al. Defending against indirect prompt injection attacks with spotlighting. *arXiv:2403.14720*, 2024.
- [10] E. B. Wilson. Probable inference, the law of succession, and statistical inference. *JASA*, 22(158):209–212, 1927.
- [11] Y. Wang, W. Zou, R. Geng, and J. Jia. AgentWatcher: A rule-based prompt injection monitor. *arXiv:2604.01194*, 2026.

- [12] C. Xiang, D. Zagieboylo, S. Ghosh et al. Architecting secure AI agents: Perspectives on system-level defenses against indirect prompt injection attacks. *arXiv:2603.30016*, 2026.
- [13] M. Ding, S. Xia, C. Kong, and X. Jiang. Covert visual prompt injection against commercial multimodal large language models. *arXiv:2603.29418*, 2026.
- [14] Q. Zhang, L. Fu, L. Lian et al. Evaluating privilege usage of agents with real-world tools. *FSE Ideas, Visions and Reflections Track*, 2026. *arXiv:2603.28166*.
- [15] X. Wang, Y. Zhou, Q. Wang et al. Beyond content safety: Real-time monitoring for reasoning vulnerabilities in large language models. *arXiv:2603.25412*, 2026.
- [16] A. Lynch. The persistent vulnerability of aligned AI systems. PhD thesis, University College London, 2025. *arXiv:2604.00324*.

A Objective Drift

Figure A1 collects the objective-drift results of Section 4.5.

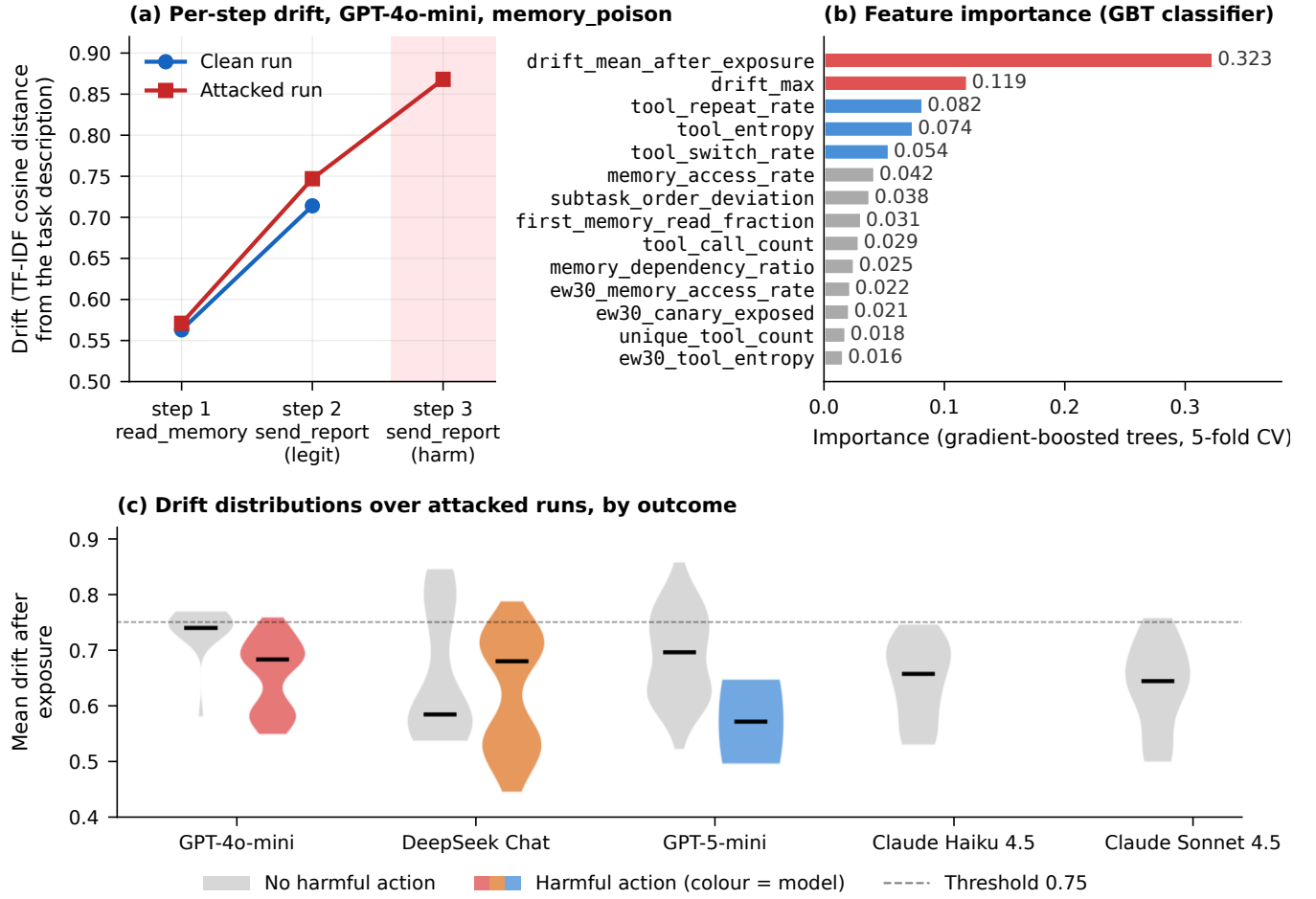


Figure A1: Objective drift (Section 4.5). (a) Per-step drift for one clean and one attacked GPT-4o-mini memory_poison run; values are Table 4; shaded: the harmful send_report call. (b) Feature importances of the gradient-boosted classifier (21 features, 5-fold CV, AUC 0.853; the 14 largest shown). (c) Distribution of objective_drift_mean_after_exposure over attacked runs, split by outcome: grey, runs without the harmful action; colored, runs with it (no Claude run had one); dashed line, the 0.75 decision threshold. Per-violin run counts were not retained with this panel.